

～ 剣 と 召 喚 の 覇 者 ～

アルカナブレイド

Arcana Blade — Complete Source Analysis

ソース解析・学習用完全技術解説書

React 19

TypeScript

Zustand

Prisma ORM

PostgreSQL

Vercel Serverless

JWT認証

PWA対応

作成日

2026年6月29日

バージョン

v0.0.0

解析対象

フルスタック RPGゲーム

目次

1. アプリ概要
2. 技術スタック
3. 全体アーキテクチャ
4. ディレクトリ構成
5. DB設計
6. ER図
7. API一覧
8. 画面・API・DB連携図
9. 主要機能別処理フロー
10. ソースファイル別解説
11. TypeScript学習
12. React / Next.js学習
13. Prisma / PostgreSQL学習
14. 認証処理学習
15. エラー・改善点
16. 学習ロードマップ
17. 今後の開発方針
18. まとめ

第1章 アプリ概要

1.1 このアプリは何か

アルカナブレイド ～剣と召喚の覇者～ は、ブラウザで遊べるスマートフォン向け **ダークファンタジー ターン制RPG** です。React + TypeScript で構築され、Vercel + Railway (PostgreSQL) でホスティングされます。PWA対応のため、スマートフォンにインストールして使うことも可能です。

1.2 ゲームとしての主要機能

機能カテゴリ	内容	実装状態
チュートリアル	タイトル→キャラ作成→バトル体験→初回ガチャまで7段階フロー	✓ 完成
バトル	ターン制・属性相性・スキル・必殺技	✓ 完成
召喚（ガチャ）	5種プール・N/R/SR/SSR排出・10連	✓ 完成
ユニット育成	レベルアップ・進化（★1～★7→★👑）・覚醒	✓ 完成
クエスト	ワールド→エリア→ステージの階層構造	✓ 完成
パーティ編成	最大5体、複数パーティ作成可能	✓ 完成
ギルド	作成・参加・チャット・ミッション	✓ 実装済（フロントのみ）
アリーナ(PvP)	ポイント制ランキング対戦	✓ 実装済（フロントのみ）
レイド	複数プレイヤーでボスを倒す	✓ 実装済（フロントのみ）
ショップ	Gold/Diamond での購入	✓ 実装済
デイリーミッション	毎日リセット・報酬獲得	✓ 完成
ログインボーナス	30日サイクル、Day30はSSR確定チケット	✓ 完成
スタミナシステム	5分で1回復、ランクアップで上限増加	✓ 完成
メール認証	emailVerifiedフィールドあり	✗ 未実装
パスワードリセット	ー	✗ 未実装

第2章 技術スタック

2.1 フロントエンド

技術	バージョン	役割	学習優先度
React	19.2.7	UIフレームワーク	★★★ 必須
TypeScript	6.0.2	型安全な開発言語	★★★ 必須
Vite	8.1.0	ビルドツール・開発サーバー	★★☆ 重要
React Router	7.18.0	SPAルーティング	★★★ 必須
Zustand	5.0.14	グローバル状態管理	★★★ 必須
Tailwind CSS	4.3.1	ユーティリティファーストCSS	★★☆ 重要
lucide-react	1.21.0	アイコンライブラリ	☆☆☆ 参考
vite-plugin-pwa	1.3.0	PWA対応	☆☆☆ 参考

2.2 バックエンド

技術	バージョン	役割	学習優先度
Vercel Serverless	@vercel/node 5.8.21	APIホスティング	★★☆ 重要
Prisma ORM	5.22.0	TypeScriptからDBを操作	★★★ 必須
PostgreSQL	(Railway想定)	リレーショナルDB	★★★ 必須
jsonwebtoken	9.0.3	JWT認証	★★☆ 重要
bcryptjs	3.0.3	パスワードハッシュ化	★★☆ 重要

第3章 全体アーキテクチャ

3.1 システム構成図



3.2 データ二重保存の仕組み

重要： このプロジェクトはゲームデータを2ヶ所に保存しています。

① **localStorage**（即時・オフラインでも動作）と ② **PostgreSQL**（1.5秒デバウンス自動保存）の二重保存です。

```
ゲームプレイ（操作）
  ↓
Zustandストア（メモリ上の状態変更）
  ↓
├─→ [即時] localStorage に自動保存
│      Zustand の persist 機能で自動管理
├─
└─→ [1.5秒デバウンス] syncService.ts
      ↓
      collectGameState() → 全ストアをJSONに
      ↓
      POST /api/player/saveAll
      ↓
      Player.gameStateJson = 全状態JSON
      + 正規化カラム (gold, diamond, etc.) 同期

ページ再読み込み時:
  GET /api/auth/me → gameStateJson取得
  → hydrateFromGameState() → 全ストア復元
```

第4章 ディレクトリ構成

4.1 全体ディレクトリマップ

arukanabrade/	
├─ api/	← バックエンドAPI (Vercel Serverless)
│ └─ auth/	← 認証関連 (register, login, me, logout)
│ └─ player/	← プレイヤーデータ (save, currency, saveAll)
│ └─ summon/	← ガチャ結果保存 (save)
│ └─ units/	← ユニット同期 (sync)
├─ lib/	← バックエンド共通ライブラリ
│ └─ auth.ts	← JWT・Cookie管理
│ └─ prisma.ts	← Prisma Clientシングルトン
│ └─ syncService.ts	← DB同期サービス
├─ prisma/	← データベース設定
│ └─ schema.prisma	← スキーマ定義 (テーブル設計)
│ └─ migrations/	← DBマイグレーション履歴
├─ src/	← フロントエンドソース
│ └─ main.tsx	← Reactのエントリーポイント
│ └─ App.tsx	← ルートコンポーネント・ルーティング
│ └─ features/	← 機能別画面 (20以上)
│ └─ stores/	← Zustand状態管理 (12ストア)
│ └─ data/	← マスターデータ (静的TS定数)
│ └─ types/	← TypeScript型定義
│ └─ utils/	← ユーティリティ関数
│ └─ hooks/	← カスタムReactフック
│ └─ lib/	← フロントエンド共通ライブラリ
│ └─ components/	← 共通UIコンポーネント
├─ package.json	← 依存ライブラリ・スクリプト
├─ vite.config.ts	← Viteビルド設定
├─ vercel.json	← Vercelデプロイ設定
└─ tsconfig*.json	← TypeScript設定 (4種類)

4.2 各フォルダの役割

フォルダ	役割	どこから呼ばれるか
<code>api/</code>	サーバー側で動くAPI。DBにアクセスして結果を返す	フロントエンドの <code>fetch()</code> から
<code>lib/</code>	複数のAPIで使いまわされる共通処理	<code>api/</code> 配下の各ファイルから
<code>prisma/</code>	DBのテーブル構造定義・マイグレーション管理	開発時コマンドで使用
<code>src/features/</code>	各ゲーム機能のReactコンポーネント群（各ページの中身）	<code>App.tsx</code> のルート定義から
<code>src/stores/</code>	アプリ全体で共有する「現在の状態」を保持	全コンポーネントから参照可能
<code>src/data/</code>	ユニット・スキル等のゲーム設定値（静的データ）	<code>features/</code> のコンポーネントから
<code>src/types/</code>	TypeScriptの型定義（全体で共有）	全ソースファイルから
<code>src/utils/</code>	汎用関数（バトルエンジン・数値フォーマット等）	<code>features/</code> ・ <code>stores/</code> から
<code>src/components/</code>	BottomNav・TopBar等の共通UIコンポーネント	<code>App.tsx</code> ・ <code>features/</code> から

4.3 features/ の画面一覧

フォルダ	URL	主な画面
auth/	/register	新規ユーザー登録
login/	/login	ログイン
tutorial/	/tutorial/*	チュートリアル7段階（タイトル〜初回ガチャ）
home/	/	メインホーム画面
units/	/units, /units/:id	ユニット一覧・詳細
party/	/party	パーティ編成
quests/	/quests	クエストマップ
battle/	/battle	ターン制バトル
summon/	/summon	ガチャ（召喚）
enhance/	/enhance	ユニット強化・覚醒
items/	/items	アイテム管理
equipment/	/equipment	装備管理
missions/	/missions	デイリー/ウィークリーミッション
raid/	/raid	レイドボス
guild/	/guild	ギルド管理
pvp/	/pvp	アリーナ（PvP）
profile/	/profile	プロフィール・称号設定
shop/	/shop	ショップ
scenario/	/scenario/:id	シナリオ（ストーリー）
debug/	/debug	開発者ツール（管理用）

第5章 DB設計

5.1 テーブル一覧

テーブル名	役割	主キー	関連
User	ログインアカウント 情報	id (CUID)	Player(1対1)
Player	ゲーム内プレイヤー データ全体	playerId (CUID)	User, OwnedUnit, PlayerItem, SummonHistory
OwnedUnit	プレイヤーが所持す るユニット	id (CUID)	Player (多対1)
PlayerItem	プレイヤーが所持す るアイテム	id (CUID)	Player (多対1)
SummonHistory	ガチャ排出履歴	id (CUID)	Player (多対1)

5.2 User テーブル

カラム名	型	制約	説明
id	String	PK, CUID自動生成	ユーザーID
email	String	UNIQUE, NOT NULL	メールアドレス (ログインキー)
passwordHash	String	NOT NULL	bcryptハッシュ化されたパスワード
emailVerified	Boolean	Default: false	メール確認フラグ (現在未使用)
createdAt	DateTime	@default(now())	作成日時 (自動)
updatedAt	DateTime	@updatedAt	更新日時 (自動)

5.3 Player テーブル

カラム名	型	制約	説明
playerId	String	PK, CUID	プレイヤーID
userId	String	FK(User.id), UNIQUE	紐付くユーザーID
playerName	String	Default: '勇者'	プレイヤー表示名（最大12文字）
tutorialCompleted	Boolean	Default: false	チュートリアル完了フラグ
playerRank	Int	Default: 1	プレイヤーランク
stamina	Int	Default: 50	現在スタミナ
maxStamina	Int	Default: 50	スタミナ上限（最大200）
gold	Int	Default: 0	ゴールド（無料通貨）
diamond	Int	Default: 0	ダイヤ（有料通貨相当）
exp	Int	Default: 0	プレイヤー経験値
title	String?	NULL可	称号
bio	String?	NULL可（最大100文字）	自己紹介文
favoriteUnitId	String?	NULL可	お気に入りユニット
loginDays	Int	Default: 1	累計ログイン日数
lastLoginAt	DateTime	Default: now()	最終ログイン日時
gameStateJson	Json	NULL可	全ゲーム状態スナップショット（重要）
createdAt	DateTime	自動	作成日時
updatedAt	DateTime	自動	更新日時

gameStateJson とは：

全ゲーム状態（所持ユニット・クエスト進捗・パーティ構成・ミッション等）をJSON形式で1

つのカラムに保存します。 ページ再読み込み時にこのJSONから全ストアを復元することで、データが失われません。

5.4 OwnedUnit テーブル

カラム名	型	説明
id	String(CUID)	インスタンスID (この1体を一意に識別)
playerId	String FK	所有プレイヤーのID
masterId	String	ユニットの「種類」ID (units.tsのIDと対応)
level	Int	現在レベル (Default: 1)
exp	Int	現在経験値 (Default: 0)
awakenRank	Int	素材覚醒段階 0~maxAwaken
awakeningCount	Int	ガチャ被り覚醒 0~4
currentRarity	String	現在レアリティ (★1~★7, ★👑)
isLocked	Boolean	ロック (売却・強化対象外)
acquiredAt	DateTime	入手日時

第6章 ER図

6.1 テーブル関係図

 User	 Player
id PK String	playerId PK String
email UNIQUE String	userId FK User.id
passwordHash String	playerName String
emailVerified Boolean	gold, diamond, exp Int
createdAt DateTime	stamina, maxStamina Int
	tutorialCompleted Boolean
	gameStateJson Json

 OwnedUnit	 SummonHistory
id PK String	id PK String
playerId FK Player	playerId FK Player
masterId String	poolId String
level, exp Int	masterId String
awakenRank Int	rarity N/R/SR/SSR
awakeningCount Int	resultType new/crystal
currentRarity String	pulledAt DateTime
isLocked Boolean	

6.2 テーブル間のリレーション

User (1) ————— (1) Player
1アカウント = 1プレイヤー
onDelete: CASCADE (Userを削除するとPlayerも削除)

Player (1) ————— (多) OwnedUnit
1プレイヤーは複数のユニットを所持
onDelete: CASCADE

Player (1) ————— (多) PlayerItem
1プレイヤーは複数のアイテムを所持
UNIQUE(playerId, itemId) で重複防止

Player (1) ————— (多) SummonHistory
1プレイヤーは複数の召喚履歴を持つ

第7章 API一覧

7.1 認証API

POST /api/auth/register — 新規ユーザー登録

項目	内容
認証必要	不要
リクエスト	<code>{ email: string, password: string }</code>
バリデーション	email: 正規表現形式チェック / password: 最低8文字
処理	<code>bcrypt.hash(pw,12)</code> → User作成 → Player作成 → JWT → Cookie
成功レスポンス	<code>{ user, player }</code>
エラー	400: バリデーション失敗 / 409: メール重複 / 500: サーバーエラー
呼び出し元	<code>src/features/auth/RegisterPage.tsx</code>

POST /api/auth/login — ログイン

項目	内容
リクエスト	<code>{ email: string, password: string }</code>
処理	DB検索 → <code>bcrypt.compare</code> → ログイン日数更新 → JWT → Cookie
成功	200: <code>{ user, player }</code>
エラー	401: 認証失敗
呼び出し元	<code>src/features/login/LoginPage.tsx</code>

GET /api/auth/me — 認証確認・ゲームデータ取得

項目	内容
認証必要	必要 (Cookie: arcana_session)
処理	JWT検証 → Player取得 (gameStateJson含む)
成功	200: <code>{ user, player }</code> (player.gameStateJson = 全ゲーム状態)
呼び出し元	authStore.checkAuth() (App.tsx起動時)

POST /api/auth/logout — ログアウト

項目	内容
処理	arcana_session Cookieを無効化 (expiredに設定)
成功	200: <code>{ ok: true }</code>
呼び出し元	authStore.logout()

7.2 プレイヤーAPI

POST /api/player/save — プロフィール保存

項目	内容
認証必要	必要
リクエスト	<code>{ playerName?, tutorialCompleted?, title?, bio?, favoriteUnitId? }</code>
バリデーション	playerName: 最大12文字 / bio: 最大100文字
処理	Player.update() (指定フィールドのみ更新)

POST /api/player/currency — 通貨・スタミナ保存

項目	内容
認証必要	必要
リクエスト	<code>{ gold?, diamond?, exp?, playerRank?, stamina?, maxStamina? }</code>
バリデーション	各値 ≥ 0
呼び出し元	バトル勝利後、チュートリアル完了後

POST /api/player/saveAll — 全ゲーム状態一括保存

項目	内容
認証必要	必要
リクエスト	<code>{ state: { ...全ゲーム状態 } }</code>
処理	gameStateJson = state (全体) + 正規化カラム同期
呼び出し元	syncService.ts の自動保存 (1.5秒デバウンス)

7.3 ゲームAPI

POST /api/summon/save — ガチャ結果保存

項目	内容
認証必要	必要
リクエスト	<code>{ poolId, units: [{masterId, rarity, resultType}], diamondSpent }</code>
resultType	<code>new</code> =新規追加 / <code>crystal</code> =被り (覚醒結晶)
処理 (トランザクション)	OwnedUnit.createMany(new only) → SummonHistory.createMany → diamond減算
呼び出し元	SummonPage.tsx, TutorialGachaScreen.tsx

POST /api/units/sync — ユニット全件同期

項目	内容
認証必要	必要
リクエスト	<pre>{ units: [{masterId, level, exp, awakenRank, awakeningCount, currentRarity, isLocked}] }</pre>
処理（トランザクション）	OwnedUnit.deleteMany（全削除）→ OwnedUnit.createMany（全再作成）
呼び出し元	TutorialGachaScreen.tsx（チュートリアル完了後）

第8章 画面・API・DB連携図

8.1 画面遷移の全体像

```

/title (タイトル画面)
├─→ /login (ログイン)
└─→ /register (新規登録)
    ↓ 認証成功
/tutorial/title → /tutorial/intro → /tutorial/name_input
→ /tutorial/hero_select → /tutorial/battle
→ /tutorial/complete → /tutorial/gacha (初回ガチャ)
    ↓ チュートリアル完了
/ (ホーム画面)
├─ /units          ユニット一覧
│   └─ /units/:instanceId ユニット詳細
├─ /party          パーティ編成
├─ /quests         クエストマップ
│   └─ /battle     バトル
├─ /summon         ガチャ
├─ /enhance        強化・覚醒
├─ /missions       ミッション
├─ /raid           レイド
├─ /guild          ギルド
├─ /pvp            アリーナ
├─ /profile        プロフィール
├─ /shop           ショップ
└─ /scenario/:id   シナリオ
  
```

8.2 認証ガードの動作

ブラウザでURLにアクセス



App.tsx が起動 → checkAuth() 実行



GET /api/auth/me



[JWT有効か?]

└─ 無効 → /title へリダイレクト ← AuthGuard

└─ 有効 → user, player, gameStateJson 取得



hydrateFromGameState(gameStateJson)

全ストアにデータ復元



[tutorialCompleted?]

└─ false → /tutorial/intro ← MainGuard

└─ true → 目的のページへ



initAutoSave() → 自動保存開始

第9章 主要機能別処理フロー

9.1 ユーザー登録フロー

```
画面: RegisterPage にメール・パスワード入力
↓
フロント: fetch('POST /api/auth/register', {email, password})
↓
API: バリデーション (形式・8文字以上)
↓
bcrypt.hash(password, 12)
↓
prisma.user.create({email, passwordHash})
prisma.player.create({userId, playerName: '勇者'})
↓
JWT生成 → Set-Cookie: arcana_session
↓
フロント: authStore.setAuth(user, player)
↓
→ /tutorial/intro ヘリダイレクト
```

9.2 ガチャ（召喚）フロー

画面: SummonPage で「10連」ボタン押下

↓

フロント: `playerStore.spendDiamond(150)` ← ダイヤ消費

↓

ガチャ計算 (クライアント側):

- ・レアリティ抽選 (SSR 4%, SR 15%, R 40%, N 41%)
- ・排出ユニットを決定

↓

`unitStore.processSummonResults(masterIds)`

└─ 新規ユニット → `OwnedUnit`追加 (`type:'new'`)

└─ 被りユニット → 覚醒結晶付与 (`type:'crystal'`)

↓

API: `POST /api/summon/save`

`{ poolId, units:[{masterId, rarity, resultType}], diamondSpent:150 }`

↓

DB: トランザクション:

`OwnedUnit.createMany` (new のみ)

`SummonHistory.createMany` (全件記録)

`Player.diamond -= 150`

↓

画面: ガチャ演出 → 結果表示

9.3 バトルフロー

クエストマップでステージ選択

↓

スタミナ消費 (playerStore.spendStamina)

↓

BattlePage初期化:

自軍: partyStore.getActiveParty() のユニット (最大5体) + フレンド

敵軍: enemies.ts のマスターデータ

↓

【プレイヤーターン】

スキル/通常攻撃選択

battleEngine.calcDamage(atk, def, power, element, element, buffs)

└─ バフ適用 → 属性相性 (最大1.5倍) → 乱数 (±10%) → ダメージ確定

敵HPを減算

↓

【敵ターン】

AIが攻撃対象選択 → calcDamage → 自軍HPを減算

↓

【バトル終了判定】

勝利:

EXP・Gold報酬付与

questStore.markCleared(stageId)

playerStore.addExp() → ランクアップ判定

POST /api/player/currency {exp, gold, ...}

敗北:

バトル結果画面

9.4 自動データ保存フロー

```
【任意のゲーム操作（gold増加・クエストクリア等）】
↓
Zustandストア更新
↓
syncService.ts の subscribe トリガー
↓
scheduleSave() ← 1.5秒デバウンス
（1.5秒以内に再操作があればタイマーリセット）
↓
collectGameState() → 全12ストアの状態をまとめてJSON化
↓
POST /api/player/saveAll { state: { player, units, quests, ... } }
↓
DB: Player.gameStateJson = state
    Player.gold = state.player.gold
    Player.diamond = state.player.diamond
    （正規化カラムも同期）
```

第10章 ソースファイル別解説

10.1 バックエンド主要ファイル

lib/auth.ts — JWT・Cookie管理

JWTの生成・検証・Cookieのセット・取り出しを担当するユーティリティファイル。複数のAPIから呼ばれる。

```
// JWTの生成（30日有効）
export function generateToken(userId: string): string {
  return jwt.sign({ userId }, process.env.JWT_SECRET!, { expiresIn: '30d' });
}

// セキュリティ設定込みのCookie設定
export function setAuthCookie(res: VercelResponse, token: string): void {
  res.setHeader('Set-Cookie',
    `arcana_session=${token}; HttpOnly; SameSite=Lax; Max-Age=2592000`
    + (isProd ? '; Secure' : ''));
}
```

学習ポイント： `HttpOnly` はJavaScriptからCookieが読めない設定（XSS対策）。 `Secure` はHTTPS通信のみCookieを送信（盗聴対策）。

lib/prisma.ts — Prisma Clientシングルトン

Prisma Clientのインスタンスを1つだけ生成して使い回す実装。開発時のホットリロードで複数インスタンスが生成されるのを防ぐ。

```
const globalForPrisma = global as unknown as { prisma: PrismaClient };
export const prisma = globalForPrisma.prisma ?? new PrismaClient();
if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma; // 開発時はグローバルに保持
}
```

src/utils/battleEngine.ts — バトルエンジン

ゲームのダメージ計算・スキル効果処理を担う。純粋関数（引数だけで結果が決まる関数）で実装されている。

```
// 属性相性テーブル（火→風は1.5倍など）
const ELEMENT_ADVANTAGE: Record< > = {
  fire: { wind: 1.5, water: 0.75 },
  water: { fire: 1.5, wind: 0.75 },
  wind: { earth: 1.5, fire: 0.75 },
  earth: { water: 1.5, wind: 0.75 },
  light: { dark: 1.5 },
  dark: { light: 1.5 },
};

// ダメージ計算
export function calcDamage(atk, def, power, atkElem, defElem, atkBuffs, defBuffs) {
  // 1. バフ/デバフを乗算
  const buffedAtk = atk * (1 + atkBuffs.reduce((a, b) => a + b, 0));
  // 2. 属性倍率
  const elementMult = ELEMENT_ADVANTAGE[atkElem]?.[defElem] ?? 1.0;
  // 3. 基本ダメージ（攻撃力×スキル倍率 - 防御力×0.5）
  const baseDmg = buffedAtk * power - def * 0.5;
  // 4. 乱数（±10%のブレ）
  return Math.floor(baseDmg * elementMult * (0.9 + Math.random() * 0.2));
}
```

10.2 Zustandストア一覧

ストア	persist key	主な状態	重要メソッド
authStore	—	user, player, isLoading	checkAuth, logout, syncCurrency
playerStore	arcana-player	player(gold/diamond/stamina), items	addGold, addExp (ランクアップ含) , recoverStamina
unitStore	arcana-units	ownedUnits, awakeningCrystals	addUnit, levelUpUnit, awakenUnit, rarityUp, processSummonResults
partyStore	arcana-party	parties (最大5枠) , activePartyId	setSlot, setLeader, getActiveParty
questStore	arcana-quests	clearedStageIds, claimedAreaRewards	markCleared, isCleared
tutorialStore	arcana-tutorial	completed, phase, selectedHeroId	setPhase, completeTutorial
equipmentStore	arcana-equipment	ownedEquipments	—
guildStore	arcana-guild	guild情報	—
raidStore	arcana-raid	raid状態	—
arenaStore	arcana-arena	record, battleHistory	—
missionStore	arcana-mission	daily/weekly進捗	—
loginBonusStore	arcana-loginbonus	lastClaimedDate, claimedDays	—

第11章 TypeScript学習

11.1 TypeScriptとは

JavaScriptに「型」を追加した言語。コードを書いた時点でエラーを発見でき、大規模なプロジェクトの保守性が向上する。

11.2 プロジェクトでの型定義 (src/types/index.ts)

```
// インターフェース (オブジェクトの型定義)
interface OwnedUnit {
  id: string;           // 必須プロパティ
  masterId: string;
  level: number;
  currentRarity: string;
  isLocked: boolean;
}

// Union型 (いずれかの文字列)
type TutorialPhase =
  | 'title' | 'intro' | 'name_input'
  | 'hero_select' | 'tutorial_battle'
  | 'initial_gacha' | 'complete';

// オptionalプロパティ
interface PlayerData {
  name: string;
  gold: number;
  title?: string;       // ? = あってもなくてもいい
  bio?: string;
}
```

11.3 TypeScriptの重要な書き方

書き方	意味	使用例
<code>string null</code>	文字列またはnull	<code>title: string null</code>
<code>prop?</code>	省略可能なプロパティ	<code>bio?: string</code>
<code>value as T</code>	型アサーション（型の強制指定）	<code>data as PlayerData</code>
<code>value ?? default</code>	nullのときdefaultを返す	<code>name ?? '勇者'</code>
<code>obj?.prop</code>	nullなら undefined を返す	<code>player?.gold ?? 0</code>
<code>value!</code>	nullではないと断言	<code>process.env.JWT_SECRET!</code>

11.4 よくあるエラーと解決策

エラー	原因	解決策
<code>Object is possibly 'null'</code>	null の可能性がある	<code>?.`</code> や <code>`??</code> を使う
<code>Type 'X' is not assignable to 'Y'</code>	型の不一致	型を合わせるか <code>as</code> で変換
<code>Property does not exist on type</code>	型定義にないプロパティ	型定義に追加する
<code>Cannot find module</code>	インポートパスが間違い	パスを確認・tsconfig確認

第12章 React / Vite 学習

12.1 Reactコンポーネントの基本

```
// 型付き関数コンポーネント
interface Props {
  unit: OwnedUnit;
  onClick: () => void;
}

const UnitCard: React.FC<Props> = ({ unit, onClick }) => {
  return (
    <div onClick={onClick} className="card bg-purple-900">
      <h2 className="text-white">{unit.masterId}</h2>
      <p className="text-gray-300">Lv.{unit.level} / {unit.currentRarity}</p>
    </div>
  );
};

export default UnitCard;
```

12.2 よく使うReactフック

フック	用途	このプロジェクトでの使用例
<code>useState</code>	コンポーネント内のローカル状態	バトル画面のターン数管理
<code>useEffect</code>	マウント時・データ変更時の処理	App.tsx の <code>checkAuth()</code>
<code>useCallback</code>	関数のメモ化（再生成防止）	ハンドラ関数の最適化
<code>useMemo</code>	計算結果のメモ化	ユニットリストのフィルタリング
<code>useNavigate</code>	プログラムの画面遷移	バトル終了後にホームへ
<code>useParams</code>	URLパラメータの取得	<code>/units/:instanceId</code> の取得

12.3 Zustandとの連携パターン

```
// コンポーネント内でZustandストアを使う
const SummonPage: React.FC = () => {
  // ストアから状態とメソッドを取得
  const { player, spendDiamond } = usePlayerStore();
  const { processSummonResults } = useUnitStore();
  const navigate = useNavigate();

  const handleSummon10 = async () => {
    if (player.diamond < 150) {
      alert('ダイヤが足りません');
      return;
    }

    // 1. ダイヤ消費（ストア更新）
    spendDiamond(150);

    // 2. ガチャ計算（クライアント側）
    const masterIds = rollGacha(10);

    // 3. ユニット処理（ストア更新）
    const results = processSummonResults(masterIds);

    // 4. DB保存（API呼び出し）
    await fetch('/api/summon/save', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ poolId: 'standard', units: results, diamondSpent: 150 })
    });
  };

  return (
    <div>
      <p>ダイヤ: {player.diamond}</p>
      <button onClick={handleSummon10}>10連召喚（150ダイヤ）</button>
    </div>
  );
};
```

第13章 Prisma / PostgreSQL 学習

13.1 Prismaスキーマの書き方

```
// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL") // 環境変数からDB接続URLを取得
}

model User {
  id          String  @id @default(cuid()) // 主キー（CUID自動生成）
  email       String  @unique              // 一意制約
  passwordHash String
  player      Player? // 1対1リレーション（0or1）
  createdAt   DateTime @default(now())     // 現在時刻がデフォルト
  updatedAt   DateTime @updatedAt          // 更新時に自動設定
}

model Player {
  playerId String @id @default(cuid())
  userId   String @unique

  // リレーション定義
  user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  ownedUnits OwnedUnit[] // 1対多（配列）

  @@index([userId]) // インデックス（検索高速化）
}
```

13.2 PrismaのCRUD操作（実際のコードより）

操作	Prismaコード	使用箇所
1件取得 findUnique	<code>prisma.player.findUnique({ where: { userId } })</code>	api/auth/me.ts
作成 create	<code>prisma.user.create({ data: { email, passwordHash, player: { create: {...} } } })</code>	api/auth/register.ts
更新 update	<code>prisma.player.update({ where: { playerId }, data: { gold: { increment: 100 } } })</code>	api/player/currency.ts
一括作成 createMany	<code>prisma.ownedUnit.createMany({ data: units })</code>	api/summon/save.ts
全件削除 deleteMany	<code>prisma.ownedUnit.deleteMany({ where: { playerId } })</code>	api/units/sync.ts
トランザクション	<code>prisma.\$transaction([op1, op2, op3])</code>	api/summon/save.ts, api/units/sync.ts

13.3 マイグレーションの使い方

```
# スキーマを変更した後に実行するコマンド

# 開発環境: マイグレーション作成+適用
npx prisma migrate dev --name "add_playerRank_column"

# 本番環境: マイグレーション適用のみ
npx prisma migrate deploy

# DBのデータをGUIで確認
npx prisma studio

# 型定義を再生成
npx prisma generate
```

第14章 認証処理学習

14.1 認証の全体像

このプロジェクトは **JWT (JSON Web Token) + HTTP Only Cookie** による認証を採用しています。

14.2 パスワードの安全な保管 (bcrypt)

```
// ❌ 絶対にやってはいけない (平文保存)
DB: { passwordHash: "password123" }

// ✅ このプロジェクトの実装 (bcryptハッシュ)
const hash = await bcrypt.hash("password123", 12);
DB: { passwordHash: "$2b$12$xxxxxxxxxxxxxxxxxxxxxxxxxx..." }

// ログイン時の照合 (元のパスワードには戻せない)
const isValid = await bcrypt.compare("password123", storedHash);
// → true (一致) or false (不一致)
```

なぜbcryptを使うのか：

通常のハッシュ (MD5・SHA1等) は計算が速いため、大量のパスワードを総当たりで試される危険があります。bcryptは意図的に遅い計算をするため、不正アクセスのコストが高くなります。`cost: 12` は計算の重さ (2¹²回の反復) を表します。

14.3 JWTの仕組み

```
// 1. ログイン成功時にJWTを生成
const token = jwt.sign(
  { userId: "clxxx123" },    // ペイロード（中身）
  process.env.JWT_SECRET,    // 署名用シークレットキー
  { expiresIn: '30d' }       // 有効期限: 30日
);
// → "eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJjbHh4eDEyMyJ9. xxxx"

// 2. 次のリクエスト時に検証
const { userId } = jwt.verify(token, process.env.JWT_SECRET);
// → { userId: "clxxx123" }
// 改ざんされていたらエラーになる
```

14.4 セキュリティ設定の意味

設定	意味	防ぐ攻撃
HttpOnly	JavaScriptからCookieを読めない	XSS（スクリプト注入）攻撃
Secure	HTTPS通信のみCookieを送信	通信傍受（中間者攻撃）
SameSite=Lax	同一サイトからのみ送信	CSRF（クロスサイトリクエスト偽造）
有効期限30日	古いトークンを無効化	盗まれたトークンの長期悪用

第15章 エラー・改善点

⚠ このドキュメントは現状の整理です。勝手な修正は行っていません。

15.1 セキュリティ（重要度：高）

#	問題	ファイル	改善案
S1	ガチャ計算がクライアント側のみ	SummonPage.tsx	サーバーAPIでガチャ計算（チート防止）
S2	Gold/Diamondをクライアントから直接送信	api/player/currency.ts	サーバー側で上限・整合性チェック追加
S3	saveAllのstateバリデーションなし	api/player/saveAll.ts	zodでサーバー側バリデーション追加
S4	JWT_SECRETがundefinedでも動作する可能性	lib/auth.ts	起動時にJWT_SECRETの存在を検証

15.2 エラーハンドリング不足

#	問題	改善案
E1	自動保存失敗時のユーザー通知なし	トースト通知で「保存失敗」を表示
E2	オフライン時のAPI呼び出し処理が不明	オフライン検知とリトライ機構
E3	ErrorBoundaryがない可能性	React ErrorBoundaryで予期しないエラーをキャッチ

15.3 DB設計改善

#	問題	改善案
D1	OwnedUnitが全削除→再作成（非効率）	upsertパターンに変更
D2	gameStateJsonが肥大化する可能性	重要データ（クエスト進捗等）は正規テーブル化
D3	quantity < 0 のチェックなし	DB Check制約の追加

15.4 未実装機能

機能	現状	優先度
メール認証	emailVerifiedフィールドあり・ロジック未実装	中
パスワードリセット	未実装	中
バトルのサーバー検証	クライアントのみ（チート可能）	中
レイドのリアルタイム	静的データのみ	低
ギルドチャットのリアルタイム	静的データのみ	低

第16章 学習ロードマップ

16.1 推奨学習順序

Step	技術	学習理由	見るべきファイル	練習課題
1	HTML / CSS	全Webの土台	src/index.css, App.css	ユニットカードをHTMLとCSSだけで作る
2	JavaScript	TypeScript・Reactの前提	src/utils/format.ts, src/data/units.ts	ユニット配列をレアルティ順にソートする
3	TypeScript	全体がTSで実装	src/types/index.ts	OwnedUnit型を見て同じ構造を型付きで作る
4	React 基礎	フロントエンド全体がReact	src/components/layout/BottomNav.tsx	ユニットカードコンポーネントを作る
5	React Router	SPA遷移の管理	src/App.tsx	App.tsxのルートを全部読んで遷移図を書く
6	Zustand	全ゲームデータを管理	src/stores/playerStore.ts	addExpのランクアップ処理を図解する
7	Serverless API	バックエンドAPIの実装	api/auth/register.ts	認証フローを図で書く
8	Prisma ORM	全DBアクセスに使用	prisma/schema.prisma, api/summon/save.ts	schema.prismaを読んでER図を手書きする
9	PostgreSQL	DBの仕組みを理解	prisma/migrations/	テーブルをSQL CREATE TABLE 文に変換する
10	JWT認証	ログイン・認証の実装	lib/auth.ts	JWTをjwt.ioで分解してみる
11	Vercelデプロイ	本番環境への公開	vercel.json, .env	Hello WorldアプリをVercelにデプロイする

Step	技術	学習理由	見るべきファイル	練習課題
12	設計書作成	ドキュメント 化能力	このドキュメント全体	改善点を3つ GitHub Issues に書く

16.2 優先度チャート

優先度	技術
● 必須（最初に）	JavaScript / TypeScript / React 基礎 / Zustand
● 重要（次に）	Prisma ORM / Serverless API / JWT認証
● 発展（余裕があれば）	PostgreSQL SQL直書き / Vercelデプロイ / PWA

第17章 今後の開発方針

17.1 短期（優先度：高）

① ガチャロジックのサーバー移行（セキュリティ）

現在はクライアント（ブラウザ）でガチャ計算をしているため、不正操作が可能な状態です。

```
// 現在（クライアント）
const masterIds = rollGacha(10); // ブラウザで計算

// 改善案
// サーバー側で計算するAPIを作成
POST /api/summon/roll { poolId, count: 10 }
→ サーバーが抽選して結果を返す
```

② APIバリデーション強化

`zod` ライブラリを導入して、APIに送られてくるデータの型・値を検証する。

③ エラー通知の実装

自動保存失敗時などにユーザーにトースト通知を表示する。

17.2 中期（優先度：中）

- メール認証の実装（`emailVerified` フィールドを活用）
- パスワードリセット機能
- テストコードの追加（バトルエンジンのユニットテストから着手）
- OwnedUnit同期をupsert方式に変更（パフォーマンス改善）

17.3 長期（優先度：低）

- マスターデータのDB化・管理画面
- WebSocketによるリアルタイム機能（ギルドチャット・レイド）
- モニタリング・ログ収集の実装

第18章 まとめ

18.1 アルカナブレイドから学べる技術マップ

学習テーマ	このプロジェクトでの実例	学習難易度
TypeScript型設計	src/types/index.ts の包括的な型定義	★★
Reactコンポーネント設計	features/ の機能ごとの分離設計	★★
グローバル状態管理	Zustand 12ストアの設計と自動保存	★★★
SPA認証	JWT + HTTP Only Cookie のセキュアな実装	★★★★
サーバーレスAPI	Vercel Serverless Functions 9本	★★
ORM活用	Prisma による型安全なDB操作	★★★★
ゲームロジック実装	battleEngine.ts の属性相性・ダメージ計算	★★
データ永続化戦略	localStorage ↔ PostgreSQL ハイブリッド保存	★★★★★
デバウンス最適化	1.5秒デバウンス自動保存	★★
Prismaトランザクション	ガチャ・ユニット同期での整合性保証	★★★★

18.2 総合評価

カテゴリ	評価	コメント
機能完成度	★★★★☆	主要ゲーム機能はほぼ実装済み
セキュリティ	★★☆☆☆	ガチャ・バトルのサーバー検証が必要
型安全性	★★★★☆	フロント側は良いがAPIの型が甘い
パフォーマンス	★★★★☆	自動保存の最適化余地あり
保守性	★★★★☆	テストなし・マスターデータが肥大化傾向
学習価値	★★★★★	モダンWebゲームの全スタックを学べる最高の教材

18.3 このプロジェクトの特徴まとめ

アルカナブレイドは「モダンなWebゲームの全スタック」を学習できる優れたプロジェクトです。

フロントエンド（React + TypeScript + Zustand）から バックエンド（Vercel Serverless）、DB設計（Prisma + PostgreSQL）、認証（JWT + bcrypt）、デプロイ（Vercel + Railway）まで、実際のプロダクションレベルの技術が一貫して使われています。

段階的に読み解くことで、現代のWebアプリ開発の全体像を把握できます。

解析日: 2026年6月29日 / 解析対象: C:\Users\y-okawa\Desktop\appsetting\arukanabrade
アルカナブレイド ソース解析・学習資料 — 完全技術解説書